

NexusAI technical documentation

Julian Calvo. AAI6850, Module 15 capstone. August 2026.

1. What it is

NexusAI is a marketplace for AI model licences that runs across several EVM chains. Three things make it different from a plain listing contract:

1. A provider has to post a bond and declare an accuracy floor. If the model later measures below that floor, the contract stops selling it.
2. The accuracy number comes from an oracle committee, not from the provider. The contract takes the median of the reports.
3. Buyers who need to prove eligibility do it with a Merkle proof of one attribute. No personal data goes on chain.

The model on the marketplace is a phishing email classifier I wrote in `ml/`. Its evaluation script writes `ml/reports/oracle_report.json` and a Hardhat script pushes that into the oracle, so the ML result is what gates the sale.

Non-goals

It isn't a compute marketplace or a model hosting platform. It also isn't audited and it isn't on mainnet.

Repo layout

```
contracts/  
  token/NexusAIToken.sol  
  staking/StakingVault.sol  
  governance/NexusGovernor.sol  
  governance/NexusTimelock.sol  
  marketplace/AIModelMarketplace.sol  
  oracle/AIPerformanceOracle.sol  
  compliance/ComplianceRegistry.sol  
  privacy/SealedBidLicenceAuction.sol  
  crosschain/CrossChainRegistry.sol  
  crosschain/MockCrossChainRouter.sol  
  interfaces/  mocks/  
test/          195 tests, 13 of them attack simulations  
scripts/       deploy, verify, governance-handoff, report-oracle, crosschain-demo  
ml/            dataset, features, training, evaluation, reports  
docs/          this file, monitoring, business case, diagrams  
audit/         security review, slither output
```

2. Architecture

See docs/diagrams/architecture.png.

The stack is roughly four layers. Token layer is NexusAIToken and StakingVault. Market layer is AIModelMarketplace and SealedBidLicenceAuction. Data layer is AIPerformanceOracle and

ComplianceRegistry. Control layer is NexusGovernor and NexusTimelock. CrossChainRegistry sits off to the side and talks to a router.

Hub and spoke

One chain is the hub (Sepolia). It has the full stack: token with the genesis mint, staking, governance, oracle, compliance, marketplace, auction, registry.

Spokes (Hoodi, Base Sepolia, Arbitrum Sepolia, Polygon Amoy) get a marketplace, a compliance registry, a cross-chain registry, and a token with zero genesis supply. All NEXA on a spoke arrives over the bridge, so the 1B cap can't be exceeded globally. Governance and staking only exist on the hub, because two governors on two chains would conflict.

3. Contracts

NexusAIToken

ERC20 with ERC20Capped, ERC20Votes, ERC20Permit, ERC20Burnable and AccessControl.

- Hard cap 1,000,000,000 NEXA, immutable
- Genesis mint 400,000,000 to the Timelock, hub only
- The other 600M can only be minted by MINTER_ROLE, which the Timelock holds
- Minting is capped per 30-day epoch by emissionCeiling, and that ceiling can never be set above MAX_EPOCH_EMISSION (12,500,000), which is a constant, so no vote can inflate past it
- Protocol fees are partly burned, which pushes supply down at volume

StakingVault

Four lock tiers: 0, 90, 180, 365 days, weighted 1.0x, 1.25x, 1.6x, 2.2x. Rewards use the Synthetix reward-per-token accumulator. Providers post their marketplace bond here, and SLASHER_ROLE (the Timelock) can cut up to 30% in one event (MAX_SLASH_BPS = 3000).

A bond has to be locked long enough to still be there through the dispute window. Otherwise a provider could stake, list, sell and unstake in the same block. That was finding H-01.

AIModelMarketplace

Holds listings, sells licences, escrows the seller's share, handles disputes.

The purchase path checks, in order: model is active, oracle score is fresh and above minAccuracyBps, circuit breaker is not tripped, buyer is compliant if the listing requires it, provider's bond is still good. Then it pulls payment, splits the fee, and escrows the rest.

Fee split at defaults is 5% protocol fee, of which 30% is burned and 70% goes to the treasury. The remaining 95% sits in escrow for a 7-day dispute window. If nobody disputes, releaseEscrow pays the provider. If the arbiter rules for the buyer, the buyer is refunded from escrow and the provider's bond is slashed.

MAX_PROTOCOL_FEE_BPS is 1000, so governance can't raise the fee past 10%.

AIPerformanceOracle

Reporters (up to 31) submit accuracy, latency and drift for a model id. The contract needs `minQuorum` fresh reports (default 3) inside `stalenessWindow` (default 1 day) before it returns a score. It publishes the median.

If a new median differs from the last by more than `maxDeviationBps` (1500), the circuit breaker trips for that model and sales stop until an admin calls `clearCircuit`. Demo step 9 shows this: the model drops from 96% to about 61% and purchases start reverting.

ComplianceRegistry

An attester writes an attestation for an address: a Merkle root of that buyer's attributes, a country code, and an expiry. Nothing else. To prove one attribute (say "accredited institution") the buyer supplies a Merkle proof for just that leaf, and only that leaf.

Admins can block a jurisdiction, deny an address, or revoke an attestation. Revocation takes effect immediately.

SealedBidLicenceAuction

Commit-reveal auction for exclusive licences. Bidders commit `keccak256(abi.encode(bidder, amount, salt))` and post uniform collateral, so the size of a bid is invisible until reveal and the deposit doesn't leak it either. Binding the commitment to the bidder's address stops someone copying a commitment they saw in the mempool.

Not revealing costs you a penalty out of collateral. Otherwise you could commit a huge bid, watch the reveals, and walk away for free.

On settle, the winner gets an entitlement through `AIModelMarketplace.issueExclusiveLicence`, gated by `LICENCE_ISSUER_ROLE`, which only the auction contract holds. That path also runs the compliance check, which it originally didn't. That was finding M-05.

CrossChainRegistry and MockCrossChainRouter

The registry publishes and receives listing and entitlement messages. The router is the transport. On local networks the router is a mock that reproduces the real shape of the problem (send on one side, execute on the other, nonces, trusted remotes) without pretending to be CCIP. `deploy.js` won't put the mock on a public network unless you set `TESTNET_DEMO_ROUTER=true`, which I did for the testnet demo.

NexusGovernor and NexusTimelock

OpenZeppelin Governor on a timestamp clock. 4% quorum, 1-day voting delay, 5-day voting period, 250,000 NEXA proposal threshold. Voting power is snapshotted at proposal time, so you can't flash-borrow votes after the fact. There's a test for exactly that.

The Timelock has a 2-day minimum delay and, after handoff, holds every privileged role in the system.

4. Trust model

After `scripts/governance-handoff.js` runs, the Timelock holds all 19 privileged roles and the deploying key holds none. The script exits non-zero if any EOA still has one, so it works as a release gate.

Three roles are never held by a person:

- `CROSSCHAIN_ROLE` on the marketplace, held only by the local registry contract
- `LICENCE_ISSUER_ROLE`, held only by the auction contract
- `SLASHER_ROLE` on the vault, held only by the Timelock

`PUBLISHER_ROLE` on the registry stays on a hot operations key. Publishing can only rebroadcast something that already exists on the source chain, and the destination reauthenticates everything anyway, so a 2-day delay on routine mirroring would cost liveness for nothing.

Things governance cannot do at all, because they're constants:

Bound	Value
Total supply	1,000,000,000 NEXA
Emission per 30-day epoch	12,500,000 NEXA
Protocol fee	10%
Slash per event	30%

`test/BranchCoverage.test.js` walks every privileged entry point and asserts an unprivileged caller is rejected.

5. Oracle

The point of the oracle is that a provider's own accuracy claim is worth nothing. The buyer can't verify it before paying and the provider has every reason to overstate it.

So reporters run the model against a holdout set the provider hasn't seen and submit accuracy in basis points. The contract takes the median rather than the mean so one extreme report cannot drag the result.

Data path:

```
ml/src/evaluate.py
-> ml/reports/oracle_report.json (modelId, accuracyBps, latencyMs, driftBps, evaluationCo
-> scripts/report-oracle.js
-> AIPerformanceOracle.submitReport
-> AIModelMarketplace.purchaseLicence reads it
```

`evaluationCommit` is a hash of the holdout manifest. It doesn't do anything on chain today, but it's what a third party would need to reproduce the score, so it's in the payload.

Reporters publish the score and the manifest hash. They don't publish the holdout set, since a public one would stop being useful.

Failure modes:

What happens	On-chain result
Fewer than quorum reports	Score unavailable, purchases revert
All reports older than the window	Same
Median jumps more than 15%	Circuit breaker trips, purchases revert
Median below the listing floor	Purchases revert, listing otherwise fine

6. Privacy and compliance

Two mechanisms.

Commit-reveal auctions. A bidder's willingness to pay is commercially sensitive. Plain on-chain bidding publishes it to every competitor. Commit-reveal with uniform collateral hides both the amount and any signal the deposit size would otherwise give away.

Merkle selective disclosure. The registry stores a root, a country code and an expiry. Everything else stays with the attester. A buyer proving they're an accredited institution reveals exactly that one leaf. Nobody learns their name, their address, or their other attributes.

On GDPR, the argument I'd make is that no personal data is written to an immutable ledger. A Merkle root of salted attributes isn't identifying on its own, and erasure is handled off chain by the attester dropping the record, plus `revoke()` on chain which takes effect immediately. I'm not a lawyer and none of this has been reviewed by one.

Enforcement points: purchase, exclusive licence issuance, auction bidding, and cross-chain mirroring all check compliance where the listing requires it.

7. Cross-chain

Envelope is `abi.encode(uint8 msgType, bytes body)`. Two types:

Type	Constant	Body
Model listing	<code>MSG_MODEL = 1</code>	<code>ModelPayload</code>
Licence entitlement	<code>MSG_LICENCE = 2</code>	<code>LicencePayload</code>

```

struct ModelPayload {
    bytes32 modelId;
    address provider;
    uint96 price;
    bytes32 weightsHash;
    string metadataURI;
    uint16 minAccuracyBps;
    uint32 licenceTerm;
    bool requiresCompliance;
}

struct LicencePayload {
    bytes32 modelId;
    address holder;
    uint64 expiresAt;
}

```

expiresAt is an absolute timestamp rather than a duration, so a message that sits in a queue for a week can't extend an entitlement by a week. And the payload carries no fee, escrow or bond state, so a compromised spoke can't make claims about hub-side money.

Anything other than type 1 or 2 reverts with `UnknownMessageType`.

The source side never reaches into the destination. `publishModel` encodes and calls `router.sendMessage`; the router assigns a nonce and emits `MessageSent`; a relayer calls `relayIn` on the destination router; that calls `ccReceive` on the destination registry.

`ccReceive` then checks three things itself, not trusting the router: the caller is the configured router, the source chain and sender match the configured trusted remote, and the nonce has not been seen before. Replaying a delivered message and calling `ccReceive` directly are both tested and both revert. Demo step 8 shows it.

Swapping the mock for CCIP or LayerZero means writing one adapter behind `ICrossChainRouter`. Everything above the interface stays as it is.

8. Deployment

Order:

1. Deploy `NexusTimelock`
2. Deploy `NexusAIToken` (genesis mint to the Timelock on the hub, zero on spokes)
3. Deploy `NexusGovernor`, wire it as the Timelock's proposer and canceller, set executor to `address(0)`
4. Deploy `StakingVault`, `AIPerformanceOracle`, `ComplianceRegistry`
5. Deploy `AIModelMarketplace`, point it at the vault, oracle and registry
6. Deploy `SealedBidLicenceAuction`, grant it `LICENCE_ISSUER_ROLE`
7. Deploy `CrossChainRegistry` and the router, grant the registry `CROSSCHAIN_ROLE`
8. Set trusted remotes both directions
9. Run `governance-handoff.js`

Commands are in the README. Run `scripts/preflight.js` first: it estimates the deploy gas against the live network and checks the balance covers it, without sending anything. Then run the handoff with

DRY_RUN=true and read the table before running it for real.

One trap on the spokes. Governance only exists on the hub, so a spoke's Timelock is constructed with no proposers. If the handoff then renounces the deployer's admin, that Timelock can never schedule anything again, and since every admin role on the spoke was just handed to it the chain is frozen for good: no pause, no unpause, no setParameters, no clearCircuit. On a chain with no Governor the script now falls back to the deployer as proposer and says so loudly, rather than leaving the timelock with none. Set SPOKE_PROPOSER to an ops multisig to override that. Either way the verification table checks the proposer really holds PROPOSER_ROLE before reporting success.

Rollback: nothing here is upgradeable, so rollback means pausing. Marketplace and vault both have pause(). Past that, you redeploy and re-point.

9. User flows

Provider. Acquire NEXA, stake at least minProviderBond with a lock that outlasts the dispute window, register the model with a weights hash and a metadata URI, then wait for reporters to establish a score.

Buyer. Get attested if the listing needs it, approve NEXA, call purchaseLicence. Licence expiry is block.timestamp + licenceTerm.

Dispute. Buyer calls openDispute inside 7 days. Arbiter calls resolveDispute. Buyer-favourable means refund from escrow, a slash of the provider bond, and delisting. Provider-favourable means escrow releases as normal.

Auction. Seller creates it, bidders commit with uniform collateral, reveal window opens, settle issues the exclusive licence to the top revealed bid, losers withdraw collateral, no-shows lose the penalty.

Governance. Hold 250,000 NEXA, propose, wait 1 day, 5-day vote, 4% quorum, queue, wait 2 days, execute.

10. Testing

195 tests across 11 files. Every fixed security finding has a named regression test, so if a fix regresses the test tells you which finding came back.

File	Focus
AIModelMarketplace.test.js	Listing, purchase gates, escrow, disputes
StakingVault.test.js	Tiers, rewards, slashing, bond locks
AIPerformanceOracle.test.js	Quorum, staleness, median, circuit breaker
ComplianceRegistry.test.js	Attestation, proofs, jurisdiction, revocation
SealedBidLicenceAuction.test.js	Commit, reveal, settle, penalties
CrossChain.test.js	Message auth, replay, mirroring, unknown types
Governance.test.js	Proposals, timelock, flash-loan resistance
Exclusivity.test.js	Exclusive licence issuance and its compliance gate
NexusAIToken.test.js	Cap, epochs, emission ceiling, votes
BranchCoverage.test.js	Access-control matrix, revert branches
Integration.e2e.test.js	Full lifecycle end to end

Coverage: 99.76% statements, 92.56% branches, 99.15% functions, 99.82% lines.

The 13 adversarial tests include a hostile ERC20 that re-enters the payment path, a rogue registry trying to inject listings, a replayed bridge message, and a post-snapshot governance takeover attempt.

CI (.github/workflows/ci.yml) runs 8 jobs. Four are hard gates: coverage below 90% fails, an unaccepted High Slither finding fails, a contract over the EIP-170 24,576-byte limit fails, and model accuracy below 90% fails.

11. Gas

Measured with `REPORT_GAS=true npx hardhat test`, optimizer on, 200 runs. Averages for the paths that matter:

Method	Avg gas
registerModel	199,712
purchaseLicence	275,690
issueExclusiveLicence	96,737
releaseEscrow	54,172
openDispute	33,283
resolveDispute	59,126
submitReport	77,152
stake	172,364
attest	95,576
commitBid	131,077
revealBid	61,665
publishModel	516,122

purchaseLicence is the expensive one because it reads the oracle, the compliance registry and the vault, then does a burn, a transfer and an escrow write.

Deployment, all under EIP-170:

Contract	Deploy gas	Bytecode	% of limit
NexusGovernor	3,949,638	17,164 B	69.8%
AIModelMarketplace	3,264,079	13,725 B	55.8%
NexusAIToken	2,344,253	9,377 B	38.2%
StakingVault	2,072,303	8,205 B	33.4%
SealedBidLicenceAuction	2,047,109	8,646 B	35.2%
CrossChainRegistry	1,760,713	7,303 B	29.7%
NexusTimelock	1,555,952	6,536 B	26.6%
AIPerformanceOracle	1,550,159	6,543 B	26.6%
ComplianceRegistry	1,019,284	4,212 B	17.1%

Hub total is about 19.6M gas excluding the mock router. NexusGovernor is the biggest and it's nearly all inherited OpenZeppelin code.

12. Known limitations

1. No cross-chain licence revocation. A buyer-favourable dispute on the hub truncates the licence locally, but a licence already mirrored to a spoke survives until it expires. Licences are short (30 days default) and mirroring is a manual operator action, which bounds it. The fix needs a monotonic revocation watermark per model and holder so out-of-order delivery can't undo a revocation. I did not have time to build that, so the gap is still there.
2. Rewards that accrue while the vault is empty are stranded. The accumulator advances `lastUpdateTime` with no stakers, so that slice never gets distributed. No one can withdraw it.
3. The protocol fee isn't refunded on a dispute. It was burned and paid at purchase time. Capped at 5%.
4. Exclusivity doesn't propagate cross-chain. A model exclusively licensed on the hub can still be bought on a spoke.
5. `releaseEscrow` reverts with `NoOpenDispute` when a dispute is open. The behaviour is right, the error name reads backwards, and renaming it breaks the interface.
6. The router is a mock. Production needs a real adapter.
7. The ML corpus is synthetic.

Next things I'd build, in order: the revocation watermark, a real CCIP adapter, Foundry invariant tests on the reward accumulator, an arbitration multisig instead of a single arbiter role, and re-running the ML on real corpora.

13. References

- OpenZeppelin Contracts 5.6.1

- Synthetix StakingRewards reward-per-token pattern
- EIP-170 contract size limit
- Chainlink CCIP and LayerZero docs, for the router interface shape
- Contracts, tests and reports in this repository